Benchmarking Fully Homomorphic Encryption Libraries in IoT Devices

Taimur Rahman
United International University
Dhaka, Bangladesh
trahman221427@bscse.uiu.ac.bd

Anas Mohammad Ishfaqul Muktadir Osmani
United International University
Dhaka, Bangladesh
aosmani203004@bscse.uiu.ac.bd

Mohammad Shahriar Rahman
United International University
Dhaka, Bangladesh
mshahriar@cse.uiu.ac.bd

Mir Moynuddin Ahmed Shibly
United International University
Dhaka, Bangladesh
moynuddin@cse.uiu.ac.bd

Salekul Islam
North South University
Dhaka, Bangladesh
salekul.islam@northsouth.edu

Abstract

Homomorphic encryption is a standard that allows arithmetic and logical operations to be performed on encrypted data. This is relevant in data analytics, where sensitive data such as health records or bank transactions cannot be shared. Many of these tasks are based on IoT devices, so integrating Homomorphic Encryption (HE) in the existing systems can ensure a reliable level of data privacy. Therefore, we have experimented with three Fully Homomorphic Encryption (FHE) libraries and accompanying schemes on the Raspberry Pi 4. Our emphasis was on analysing other factors such as temperature and RAM usage accounting for the resource constrained device. In the fast evolving realm of FHE libraries supporting various schemes, this study has given an insight to the current standings of the most common libraries in terms of computation efficiency and implementation optimisation. We have found that OpenFHE library has a better implementation of the BFV scheme with regards to CPU usage and server-side arithmetic operations, being on average 50.4% better than the second best library. SEAL has outperformed the other libraries with a relatively lower temperature profile and faster arithmetic on BFV and CKKS schemes. On the other hand, key generation is far better on Lattigo, being on average 12.7 times faster than other libraries.

CCS Concepts

• **Security and privacy → Cryptography**.

Keywords

homomorphic encryption, data-privacy, benchmarking, IoT

ACM Reference Format:
Taimur Rahman, Anas Mohammad Ishfaqul Muktadir Osmani, Mohammad Shahriar Rahman, Mir Moynuddin Ahmed Shibly, and Salekul Islam. 2024. Benchmarking Fully Homomorphic Encryption Libraries in IoT Devices. In *11th International Conference on Networking, Systems, and Security (NSysS '24), December 19–21, 2024, Khulna, Bangladesh.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3704522.3704546

NSysS '24, December 19–21, 2024, Khulna, Bangladesh

ACM ISBN 979-8-4007-1158-9/24/12
https://doi.org/10.1145/3704522.3704546

1 Introduction

Computation is moving to more distributed systems, with two common paradigms in cloud computing and edge computing. IoT edge devices have become ubiquitous and widespread in usage, but their limited computation power often means cutting back on privacy measures. Botnet attacks, such as the Mirai botnet[1], have targeted these vulnerable devices. On the contrary, currently employed security measures in the existing cryptosystems are at risk of being exploited by quantum computers. As a response to mitigate this risk, the field of post-quantum cryptography is continuously expanding in recent years [5]. Among the various approaches considered to be resilient against advanced quantum algorithms, lattice-based cryptography is a well-received method. Homomorphic Encryption (HE) falls under the lattice-based cryptography. In addition, Fully Homomorphic Encryption (FHE) is considered to be the holy grail of cryptography as it allows arbitrary computation on encrypted data without any upper bound for the number of operations. Ever since the publication of the first FHE scheme [13], the notion of FHE being infeasible due to the shear computational cost and the impractical time required to perform basic operations on encrypted data remained vastly unchanged in the research community.

However, FHE has evolved rapidly in the past few years as more schemes are invented that are significantly better than the initial FHE scheme [14]. Furthermore, these schemes are now made accessible in the form of open-source libraries that implements one or more of the FHE schemes. This has opened the possibility of building secured applications that integrates FHE allowing services involving a distrustful server to be deployed. While different schemes are good for specific use cases, it would be worthwhile to find out the efficacy of the schemes with respect to the libraries used on lower end hardware. Particularly, we have been interested in the some of the schemes that have been selected to be standardised; namely Brakerski/Fan-Vercauteren (BFV) [17], Brakerski-Gentry-Vaikuntanathan (BGV) [18] and Cheon-Kim-Kim-Song (CKKS) [19] by the ISO/IEC as mentioned in [6]. In our case, we have conducted the experiments in a Raspberry Pi 4 Model B; an IoT device widely used both in research and industrial applications.

[1] https://www.cloudflare.com/learning/ddos/glossary/mirai-botnet/

Besides, all FHE schemes require setting several parameters that can affect the performance and security of the system. Aside from the time taken for different operations (addition, multiplication, rotation), there are additional results that are crucial to consider, such as the CPU temperature or power draw or RAM usage when it comes to resource constrained devices. Learning this may provide insight on possible side-channeling attacks [15].

For interconnected security systems, encrypted data needs to be communicated over a network. The process of turning encrypted data into a bytestream is serialisation. The time it takes for serialisation as well as the size of the ciphertext is crucial for networks. The size of the ciphertext determines the communication overhead, therefore federated learning systems require effective serialisation. It can be valuable to find a library that can produce ciphertext with the minimum size allowing for a smaller communication overhead.

The following is a summary of contributions of this paper:

- **C1:** Compare the execution time of basic homomorphic operations using three FHE schemes implemented in three commonly used open-source libraries namely OpenFHE, SEAL and Lattigo.
- **C2:** Measure hardware-related external factors while executing the test case for the different schemes in each libraries.
- **C3:** Explore how each FHE libraries handle serialisation and compare the execution time and the resulting ciphertext size per library per scheme.

2 Preliminaries

2.1 Glossary

Term	Definition
Plaintext	Readable information in a natural form, accessible and usable.
Ciphertext	Safeguarded plaintext through the means of encryption, to prevent unauthorized access.
Bootstrapping	A decryption procedure that "refreshes" a ciphertext with a lot of accumulated noise with an "equivalent" ciphertext with less noise.
Serialization	Transforming a data structure into a byte stream that can be stored, transmitted, and reconstructed.

Table 1: Glossary of Terms

2.2 Internet of Things

Internet of Things is a term referring to the network of physical devices and embedded systems whose connectivity allows them to exchange data, connecting everyday objects to the Internet. These devices have evolved to become smaller, more cost effective, and energy efficient. This comes at the cost of limited computational capabilities, which adds significant challenge to implement both complex algorithms, network communication; and most important yet overlooked; security mechanisms. [27] lists several obstacles such as inter-connectivity and heterogeneity, along with resource constraints.

A major function of IoT devices is routing which led to the creation of the RPL (Routing Protocol for low power and lossy network) designed specifically to be used with IoT devices. However, this protocol is vulnerable to a number of cyber attacks like Sybil attack, sinkhole attack, selective forwarding attack and wormhole attack. This are some of the prominent network layer attacks. A survey was conducted in [20] on detection and prevention of sinkhole attacks. For evaluating the preventive measures, performance matrices like packet delivery ratio, throughput, threshold value, scalability and energy consumption were considered.

Cyber attacks in IoT device can occur in different layers through which the data passes by; perception layer, network layer, middleware layer and application layer. Attacks in the perception layer includes tag cloning, eavesdropping, RF jamming and spoofing attack etc. A review of this layer-wise attacks and their counter measures was conducted in [4]. The network layer is the primary target for attacking IoT device. In addition to the aforementioned network layer attacks, there is DoS attack, DDoS attack, Spoofed Routing Information attacks and Man-in-the-Middle attack etc. Then, the middleware layer attacks include the Flooding attack, Cloud Malware Injection, SQL Injection etc. For the application layer code injection, buffer overflow and Phishing attacks are some of the mentionable cyber attacks.

Resource limited devices have had lightweight cryptography designed specifically for their constraints. [30] reviews several lightweight algorithms both on a hardware and software level separately; comparing memory usage, latency and throughput, energy efficiency and parameters such as key size and block size. It also reviews security analysis, vulnerabilities and attacks on said devices.

2.3 Homomorphic Encryption

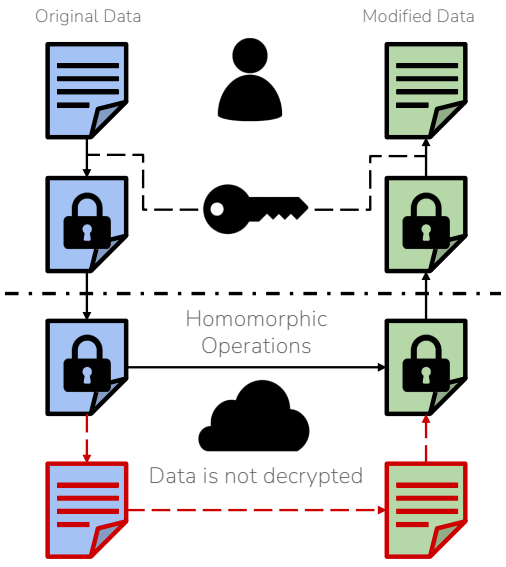

Figure 1: Overview of homomorphic encryption

Homomorphic encryption (HE) allows arithmetic operations to be performed on ciphertext. Upon decryption, the result would be the same as if it were done on plaintext. Figure 1 depicts the simplified workflow of HE. This has definite advantages over standard encryption algorithms, with a major component being cloud computing and machine learning. Cloud computing is vulnerable to data breaches, and with HE, cloud platforms such as medical institutes or banks could analyse data without infringing user privacy.

Data collected by IoT devices are often transmitted without encryption[2], and since these are connected to a network, they can introduce this vulnerability to other devices. The benefit of HE over more lightweight solutions would be that cloud providers would never have access to unencrypted data. HE primarily focuses on confidentiality by ensuring data can be securely analysed. Integrity is the most important aspect when considering IoT devices, and HE can reduce the risk of unauthorised tampering or data access.

HE can be divided into three broad categories, partially homomorphic, somewhat homomorphic and fully homomorphic. The differences between these lie in what kinds of operations can be done and how many of those operations can be done. Common fully homomorphic encryption schemes are: CKKS for floating points, BGV and BFV for integers, and Chillotti-Gama-Georgieva-Izabachène [9] (CGGI) for logical statements. Table 2 defines the parameters for BGV, BFV and CKKS used in key generation, encryption and decryption.

R_2	uniform random distribution to sample polynomials in {-1, 0, 1}
χ	error distribution
R_q	uniform random distribution to sample polynomials in {0,1,...,q-1}
SK	secret key, random ternary polynomial from R_2
PK	public key, pair of polynomials PK_1PK_2
a	random polynomial in R_q
e	random error in χ
q	positive integer used for modulo operation
t	scale factor
u	three small random polynomials from R_2
Δ	quotient from dividing q by t
C	ciphertext (C_1, C_2)
M	plaintext

Table 2: Parameters for FHE

BGV handles integers. Noise levels need to be kept track of at every step of the algorithm. Both BGV and BFV have very slow bootstrapping procedures due to the amount of key switching operations required [12], however they are both typically used as a leveled homomorphic encryption scheme. Leveled schemes only allow a certain amount of operations until it fails [7].

Key Generation

$$PK_1 = [-1(a \cdot SK + t \cdot e)]_q$$
$$PK_2 = a$$

Encryption and Decryption

$$C_1 = [PK_1 \cdot u + t \cdot e_1 + M]_q$$
$$C_2 = [PK_2 \cdot u + t \cdot e_2]_q$$
$$M = [[C_1 + C_2 \cdot SK]_q]_t$$

BFV handles integers. There are slight differences in the underlying operations that differentiate it from BGV, but are fairly identical. It is possible to losslessly switch between the two encodings [22]. The key difference between BGV and BFV is that BGV corresponds to least significant bit message encoding and BFV uses the most significant bit.

Key Generation

$$PK_1 = [-1(a \cdot SK + e)]_q$$
$$PK_2 = a$$

Encryption and Decryption

$$C_1 = [PK_1 \cdot u + e_1 + \Delta M]_q$$
$$C_2 = [PK_2 \cdot u + e_2]_q$$
$$M = \left[\left\lfloor \frac{t\,[C_1 + C_2 \cdot SK]_q}{q} \right\rceil\right]_t$$

CKKS handles floating-point numbers and is desirable for machine learning [23]. Overall, it is similar to the core operations with integer schemes, but requires an additional scaling factor that determines it's precision. After multiplying ciphertexts, the size of this scale exponentially increases, so it requires rescaling.

Key Generation

$$PK_1 = [-a \cdot SK + e]_q$$
$$PK_2 = a \xleftarrow{U} R_q$$

Encryption and Decryption

$$C_1 = [PK_1 \cdot u + e_1 + M]_q$$
$$C_2 = [PK_2 \cdot u + e_2]_q$$
$$M = [C_1 + C_2 \cdot SK]_q$$

3 Related Works

In recent years, various approaches for implementing FHE in IoT have been proposed due to the rapid advancement in optimised FHE libraries. Table 3 depicts the research gap that this work attempts to address. The last column portrays our contributions, and we have justified our reasons for not addressing some of them later in this section.

The relevant literature for benchmarking different homomorphic encyrption libraries with respect to the schemes have constantly been updating due to the formation of new libraries as well as the discontinuation of old ones. [16] proposes a standardised benchmark suite that compares between fully-homomorphic encryption (FHE) libraries (Lattigo, TFHE, SEAL, Palisade, HElib). Experiments were run on an Intel Core i7 - 11850H, Ubuntu LTS 20.04. [21] compares major FHE schemes from 6 libraries (SEAL, Palisade, HElib, CKKS, FHEW, TFHE). Due to each library implementing schemes differently, the study primarily looked at how they performed with respect to each scheme. Experiments were run on Intel Xeon Gold 6138P CPU and 64GB DRAM. Despite benchmarking numerous

[2]https://www.csoonline.com/article/567281/over-90-of-data-transactions-on-iot-devices-are-unencrypted.html

Contribution \ Related Work	[26]	[25]	[21]	[16]	Ours
Library					
OpenFHE / PALISADE		✓	✓	✓	✓
SEAL	✓	✓	✓	✓	✓
Lattigo				✓	✓
HELib	✓		✓		
TFHE			✓		
Schemes					
BFV	✓	✓	✓	✓	✓
BGV	✓	✓	✓	✓	✓
CKKS	✓	✓	✓	✓	✓
CGGI			✓	✓	
Operations					
Key Generation			✓		✓
Encryption	✓	✓	✓		✓
Addition	✓	✓	✓	✓	✓
Multiplication	✓	✓	✓	✓	✓
Rotation		✓	✓	✓	✓
Decryption	✓	✓	✓		✓
Bitwise Operations			✓	✓	
Bootstrapping			✓	✓	
Serialisation					✓
Metrics					
CPU Usage	✓				✓
Power Consumption	✓				✓
RAM Usage	✓				✓
Temperature					✓
Device					
Low Resource Devices	✓	✓			✓
Capable Devices			✓	✓	

Table 3: Gap Analysis

libraries, the efficiency of both if this benchmark suites [16, 21] in lower end hardware remained uncovered.

A feasibility analysis of implementing different HE schemes on IoT device was conducted in [26]. For the IoT device the Raspberry Pi 4 was considered and to implement the FHE schemes two well known libraries were used namely SEAL and HElib. It was found that 70.07% was the maximum energy consumption reached across the libraries and schemes which were tested. In addition, a guideline suggesting the use of multi-threading and edge computing for further optimizing HE suitable for IoT device was also provided. Nonetheless, some of the HE operations were not considered like key generation and rotation. Even though some of the external metrics were measured, the CPU temperature was not considered.

In [25], the implementation of common homomorphic encryption schemes; BGV, BFV and CKKS; on a Raspberry Pi 4 device, with two libraries in Palisade and SEAL are evaluated. They settled with a specific ciphertext dimension of $n = 8192$ and 128-bit security level. It also shown that even on the same CKKS scheme, secret key decryption on Palisade takes significantly longer compared to SEAL, but addition and multiplication are faster. However, external factors related to the hardware was not measured and similar to [26] they excluded key generation time.

We have aimed to address most of the limitations of our related work which includes testing the Lattigo library in a lower end hardware, measuring CPU temperature for executing the FHE operations and benchmarking the serialisation of ciphertext in each library. Despite covering most of the gaps determined in Table 3, we have only experimented with three of the FHE libraries because the other libraries do not have implementation of all three selected schemes. We have attempted to use the TFHE library that implements the CGGI scheme. However, due to the lack of the AES aarch64 CPU hardware feature in Raspberry Pi the code did not executed, which meant bitwise operations have also not been also not performed. As of writing this paper, the HElib library is going through extensive refactoring so we have not selected it. The bootstrapping operation is not supported in the selected libraries for all three schemes so it was decided against comparison.

4 Methodology

Figure 2 portrays the methodology we have followed. Three common FHE schemes are considered, two integer based schemes in BGV and BFV, and one floating point based scheme in CKKS. These schemes are implemented on the libraries: OpenFHE [10], SEAL [28] and Lattigo [3]. We have used the Raspberry Pi 4 Model B [1] as our testbed. We have conducted two types of experiments; one involving basic FHE operations and the other deals with serialisation of ciphertext using the three libraries. For the first type of benchmarking experiments we have measured metrics such as operation execution time, power draw, memory usage, CPU utilisation and temperature using command line operations to remain language agnostic and have our results be uniform. This is helpful because OpenFHE and SEAL are written in C++ but Lattigo is written using the Go programming language. Therefore, the same bash script containing the metric commands can be used for all the experiments regardless of the programming language. The second benchmarking experiment covers serialisation. We have developed custom test cases for both of these experiments in Section 4.4.

4.1 Experimental Setup

We have used the 64-bit Raspbian OS in a SD card with 32 GB storage. The device is remotely accessed through Secure Shell protocol(SSH). The libraries OpenFHE and SEAL require the CMake toolkit alongside ninja. CMake is a standard for building C and C++ code, and is necessary for cross-platform builds. Ninja is a build generator that allows us to quickly rebuild our executable without needing to build from scratch. It is helpful when our only change comes from the code. We have installed the CMake toolkit version 3.5.1 and Ninja version 1.5 for our experiments. As for the experiments on the Lattigo library we have developed the test cases using version 1.22.4 of the Go programming language. The specific version of the FHE libraries are mentioned in section 4.2

The hardware specification of the Raspberry Pi 4 Model B that we have used is depicted in Table 4.

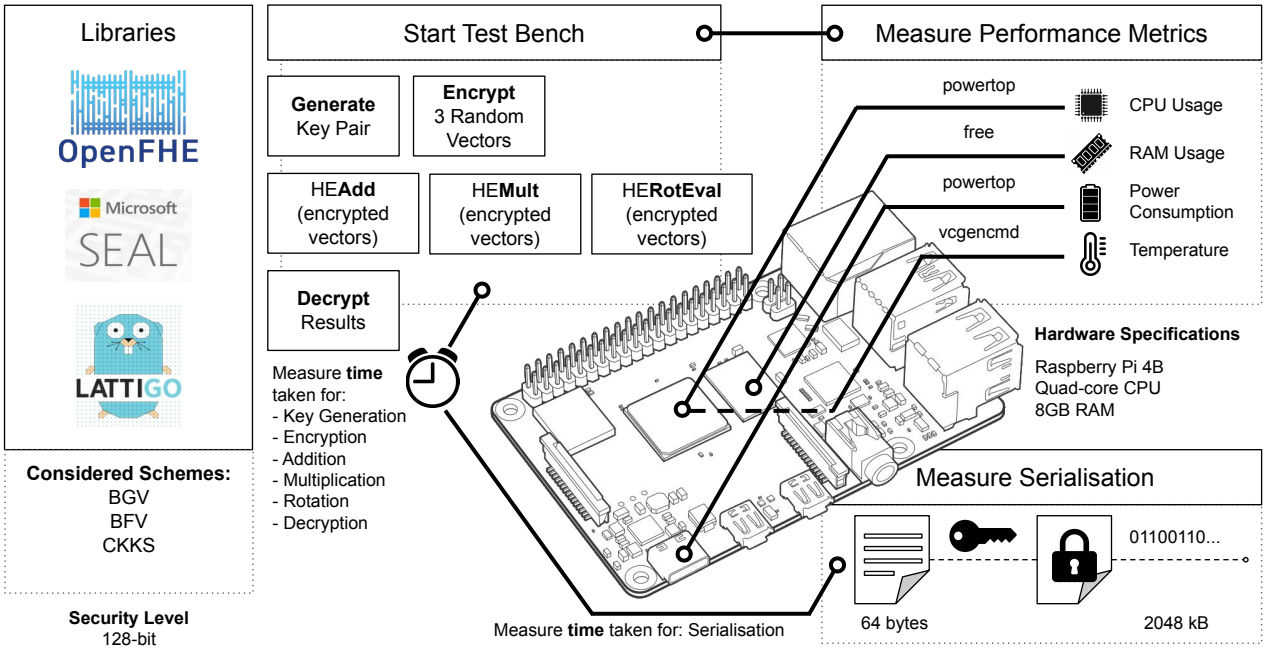

Figure 2: Proposed Methodology for our Experimental Setup

Entity	Specification
Processor	Quad-core Cortex-A72 (ARM v8) 64-bit SoC @ 1.8GHz
RAM	8 GB
Power consumption	2.5 A
Operating temperature	0 - 50 degrees C ambient

Table 4: Raspberry Pi 4 Model B specifications

4.2 Libraries

We have selected three libraries for our experiment, that are still actively maintained on Github. Other considered libraries such as HELib and HEANN have been archived or had features merged into other libraries such as OpenFHE. The libraries we have considered are summarised as follows:

4.2.1 OpenFHE. is a successor to Palisade which has been used as a library for most benchmarks. While we have not expected any major differences between OpenFHE and Palisade in terms of performance, some features from HElib and HEANN were incorporated in OpenFHE. We have experimented on version 1.1.4 of the OpenFHE library.

4.2.2 SEAL. is a library developed by Microsoft that is also used as a standard FHE library. There is also a version with a smaller memory footprint in SEAL Embedded. In addition, a Python wrapper for this library called TenSEAL is widely used to integrate FHE in python programs. We have used version 4.1 of the SEAL library.

4.2.3 Lattigo. is a library written entirely in Go for easier cross-platform usage. Even though this library is not as widely used as the other two, since it is written in a procedural programming language; Go, it may show a varying performance than the other libraries. The version of Lattigo we have used is 5.0.2.

4.3 Scheme Parameters and Security Levels

Each FHE scheme has a defined set of parameters that reflects the security level of the cryptosystem. It is based on the hardness of the underlying Learning with Error (LWE) or Ring Learning with Error (RLWE) problem. However, the FHE libraries in general have varying implementation of the same scheme. The library specific parameters automatically choose scheme parameters to make equivalent cryptosystems. Nonetheless, the security level is the common trait across all the FHE schemes regardless of the library. In [6], a guideline for selecting parameters to ensure certain security levels have been proposed. We have used the default parameters suggested by the respective libraries that provided a security level of above 128 bits which is a common choice in many of the related works [16, 21, 25]. Furthermore, we have used a multiplicative depth of 2 for all experiments. This set of parameter selection is an attempt to strike a balance between performance and careful resource utilization of the resource-constraint Raspberry Pi.

4.4 Developing a Custom Test Case

Throughout our libraries, our test involves the following Algorithm 1, where data is a vector of dimension 1000 with each element being a random number (float for CKKS, integer for BFV and BGV) from

0 to 1000. After beginning our logs for our metrics as covered in Section 4.5, we have generated a key pair and encrypted data 1, 2 and 3 (all 1000-dimensional vectors). We have then performed addition, multiplication and rotation tasks, and finally decrypt our results and end the log. One small caveat, the CKKS test bench only multiplied two vectors due to the exponential noise increase. For the serialisation we have taken a plain vector, encrypted it and serialised it. In the meantime, we have tracked the time it takes to perform serialisation. Finally, the ciphertext size and the serialisation time are printed.

Algorithm 1 Test Bench

```
procedure OPERATION_BENCH
    START_LOG
    keyPair ←GENERATE_KEYPAIR
    n ← 3
    for i ← 1 to n do
        data_i ← RANDOM(1000)
    end for
    for i ← 0 to n − 1 do
        cipher[i] ← ENCRYPT(data[i], keyPair.P_k)
    end for
    add ← HEADD(cipher[1...n])
    mult ← HEMULT(cipher[1..n])
    rotate ← HEROTATE(cipher[1])
    DECRYPT(add, mult, rotate)
    END_LOG
end procedure
procedure SERIALISATION_BENCH
    plaintext ← vector
    ENCRYPT(plaintext)
    START_TIME
    SERIALISE(ciphertext)
    END_TIME
    PRINT(serialisedsize, time)
end procedure
```

4.5 Evaluation Metrics

We have measured the execution time for the following operations: key generation, encryption, addition, multiplication, rotation, decryption and serialisation. We have also decided to measure CPU usage, RAM usage, power draw and temperature; by using command line tools that can be run independently of the library. This makes our logging language-agnostic, so it can be run against any library implemented on any language. However, some commands such as `vcgencmd` and `powertop` are device-specific, so some devices may not have the commands that are specific to the Raspberry Pi.

The runtime for our tests range within a second, therefore we have reported on a single value for our metrics. We have considered the following external factors:

- **CPU Temperature** can increase with computational load, and may even effect performance. Heat signatures have also been used in side-channel attacks. The in-built `vcgencmd` command is a Raspberry Pi utility that can be used to access the internal sensors of the device[11]. More specifically, we have used `measure_temp` to find out the temperature at the current timeframe.
- **CPU Utilisation** defines how many cores the process will take up, allowing other processes to run in parallel. The Raspberry Pi 4 is a quad-core device, and a diagnostic tool called `powertop` can monitor the performance of power draw alongside the CPU utilisation and save results into a CSV.
- **RAM Usage** is another insight to the computational load attributed to HE. In-built `free -m` is a Unix command to find show RAM usage across Mem and Swap.
- **Power Draw** is another primary factor for side-channel attacks, particularly of interest to more sustainable systems. The CLI command `powertop` will allow us to get meaningful estimates for the power draw on the Raspberry Pi.
- **Serialisation** can be observed with the time taken to serialise as well as the size of the resulting ciphertext bytestream. We have discussed the specifics on analysis in Section 5.2

5 Results and Discussion

5.1 Analysing basic FHE operations

		HE Library		
Scheme	Metric	OpenFHE	SEAL	Lattigo
BFV	CPU Usage (%)	**27.6**	60.8	119
	RAM Usage (MB)	228	306	**226**
	Power Draw (W)	4.9	**4.81**	4.94
	Temperature (°C)	43.4	**41.3**	42.3
BGV	CPU Usage (%)	**24.4**	66.5	124
	RAM Usage (MB)	249	303	**222**
	Power Draw (W)	5.22	**4.32**	5.74
	Temperature (°C)	**42.8**	43.3	43.3
CKKS	CPU Usage (%)	**42**	48.8	129
	RAM Usage (MB)	**242**	269	263
	Power Draw (W)	5.36	5.74	**5.15**
	Temperature (°C)	49.1	**42.8**	43.8

Table 5: Test Bench Results

Figure 3 and Table 5 details our findings based on our test bench where we have measured the execution time for each FHE operation and other external factors including CPU Usage, RAM Usage, Power Draw and Temperature respectively. We have extensively analysed these results from two perspectives; library-wise and scheme-wise. Furthermore, we have analysed which operations are useful in which scenarios. The reported results are the averaged values after running the experiments 20 times.

First of all, looking at Figure 3, we have made conclusions on which library performs better per FHE operation, and per scheme. Based on the graphs, it is apparent that the overall BFV implementation in OpenFHE is better than the other two libraries. When comparing with SEAL, it only takes longer with encryption, but the rest of the BFV operations are faster. Nonetheless, SEAL excels in the BGV and CKKS schemes, having excellent results across the board in each operation. Even though Lattigo performs mediocre for all three schemes, it always achieves the minimum key generation

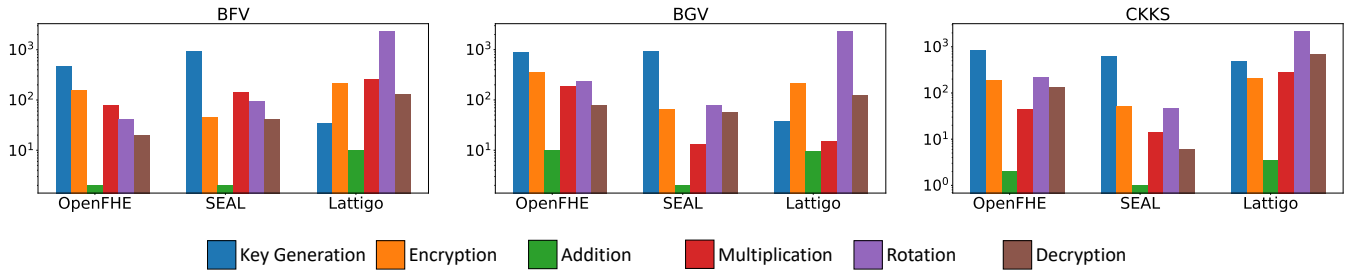

Figure 3: Comparison of FHE operations for BFV, BGV, and CKKS schemes with time (ms) on the y-axis in logarithmic scale

time with respect to the other libraries. However, the rotation operation is not as optimised, taking almost double the time compared to the other libraries across all three schemes. Lattigo is written in Go, and SEAL and OpenFHE are written in C++; and the difference in programming language shows a considerable difference in the values obtained as well.

Secondly, given the fact that both BFV and BGV schemes are used for integers we have determined which one can be more effective when it comes to choosing an integer-based FHE scheme. It is difficult to say if one scheme is preferable over the other as each one is better for different operations. The BFV scheme takes less time for key generation, encryption and rotation. On the other hand, BGV scheme takes significantly less time to multiply, matching results with related literature [25]. The addition operation takes the same time for both schemes. As for the external factors, the BGV scheme is slightly better than the BFV scheme because of lower CPU usage, RAM usage, Power consumption. Therefore, the BGV scheme can be recommended if the priority is to be conservative of the hardware resource available and if there is a demand for the multiplication operation. In contrast, if faster encryption and rotation is of importance, BFV can be a more suitable option. Through mod-switching, it is also possible to swap between BGV and BFV schemes losslessly [22]. Alternatively, if there is the option to choose between integer and floating-point FHE, the integer-based FHE can be a better choice given the fact that CKKS takes more time to perform nearly all operations except for addition and decryption. Noticeably, the decryption time for CKKS is 70% and 89% less than BFV and BGV schemes respectively.

Thirdly, each of these operations work in different parts of a network, whether it is client sided or server sided. Key generation can be handled by a key distribution server, or through threshold key distribution [8, 24]. If the shared computation between clients for generating key pairs is considered, Lattigo achieves very fast results, at least for a single client. As for client side operations, we have mainly focused on encryption and decryption, as all the arithmetic is handled server side. In this sense, SEAL outperforms the other libraries. The main arithmetic operations in addition, multiplication and rotation also vary from library to library and within schemes. OpenFHE handles BFV server side operations well, and SEAL is preferred for BGV and CKKS operations.

		HE Library		
Scheme	Metric	OpenFHE	SEAL	Lattigo
BFV	Plaintext (bytes)	64	64	64
	Ciphertext (kB)	1024	**432.4**	2150
	Serialisation Time (ms)	5.89	24.7	**4.98**
BGV	Plaintext (bytes)	64	64	64
	Ciphertext (kB)	1638	**432.4**	2150
	Serialisation Time (ms)	8.52	31.0	**4.89**
CKKS	Plaintext (bytes)	64	64	64
	Ciphertext (kB)	787.6	**334.5**	2150
	Serialisation Time (ms)	4.80	29.2	**4.17**

Table 6: Serialisation Results

5.2 Analysing serialisation

Table 6 outlines the results for the serialisation experiments. The time taken to serialise and the ciphertext size are reports for a plaintext size of 64 bytes. Each library handles serialisation differently which impacts both the ciphertext size and time needed to serialise. OpenFHE uses the `serializeToFile()` function to serialise the ciphertext and save it in a text file. SEAL has a simple `save()` method that belongs to the `Encryptor` class that saves the ciphertext as a bytestream in the given stream variable which in our case is `stringstream`. Finally, we have used the `MarshalBinary()` standard library function [2] to serialise in Lattigo since it does not have its own serialisation function. This is why the size is so large, as the function serialises the entire object file. This standard approach is also why Lattigo takes the least time for serialisation for all the experiments. On the contrary, SEAL always produced the smallest sized ciphertext, followed by OpenFHE.

6 Limitations

Despite the advantages on using fully homomorphic encryption, there are still considerable downsides that stop it from widespread use. Performance and efficiency are one of the major limitations holding FHE back, since it is still a computationally intensive process despite its leaps and strides. This may not affect smaller amounts of data, but working in scale with large amounts will quickly add up. This is exacerbated by the large key sizes required by FHE, meaning key distribution, bootstrapping and security become difficult to manage.

Another overlooked aspect of FHE's practicality is its ease of implementation, which despite regularly maintained libraries and tutorials, still are difficult to grasp. SEAL is considered to be easy to use but lacks finer control of algorithms such as OpenFHE [29]. OpenFHE is a lot more customisable but requires a lot more expertise. Lattigo is written in Go, which may be foreign to most developers. Performance degrades drastically after several operations, with accumulated noise making data unreadable after a while. Choosing the FHE parameters that provides a large enough noise budget while keeping the ciphertext size to a minimum can be challenging.

7 Future Work

There are other FHE libraries out there aside from the three that have been tested. Other libraries such as HELib, HEANN as well as bitwise libraries such as TFHE may also be considered. A limitation of our work was only operating on floating point and integer evaluations only, and binary evaluations should also be considered. More realistic benchmarks like linear regression and neural network inferences can provide more insights on the feasibility of FHE. Working on physical hardware provides actual tangible values, so another avenue for future work would be to test FHE on other low resource hardware.

8 Conclusion

In this paper, we have ran tests on three different FHE libraries on a resource limited device to observe their performance and impact on the device. We have found that for different operations different libraries are better optimised. We have concluded that the overhead that FHE adds in isolation is not overbearing for resource constrained devices. However, when in more intensive work loads, such as within a neural network, the time taken would increase exponentially. More exhaustive operations such as bootstrapping should also be considered in the future, which would give insights to prolonged usage of FHE schemes.

We believe our work will assist in making decisions on which library and scheme to be used in low resource hardware, as well as point out where there may be bottlenecks or considerations.

References

[1] 2019. Raspberry Pi 4 Model B Specifications. https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/.

[2] 2022. Marshal and Unmarshal Binary Formats in Go. https://github.com/encodingx/binary.

[3] 2023. Lattigo v5. Online: https://github.com/tuneinsight/lattigo. EPFL-LDS, Tune Insight SA.

[4] Ghaida Alqarawi, Bashayer Alkhalifah, Najla Alharbi, and Salim El Khediri. 2023. Internet-of-things security and vulnerabilities: case study. *Journal of Applied Security Research* 18, 3 (2023), 559–575.

[5] Daniel J Bernstein and Tanja Lange. 2017. Post-quantum cryptography. *Nature* 549, 7671 (2017), 188–194.

[6] Jean-Philippe Bossuat, Rosario Cammarota, Jung Hee Cheon, Ilaria Chillotti, Benjamin R Curtis, Wei Dai, Huijing Gong, Erin Hales, Duhyeong Kim, Bryan Kumara, et al. 2024. Security Guidelines for Implementing Homomorphic Encryption. *Cryptology ePrint Archive* (2024).

[7] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2014. (Leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)* 6, 3 (2014), 1–36.

[8] Hao Chen, Ilaria Chillotti, and Yongsoo Song. 2019. Multi-Key Homomorphic Encryption from TFHE. In *Advances in Cryptology – ASIACRYPT 2019*, Steven D. Galbraith and Shiho Moriai (Eds.). Springer International Publishing, Cham, 446–472.

[9] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. 2020. TFHE: fast fully homomorphic encryption over the torus. *Journal of Cryptology* 33, 1 (2020), 34–91.

[10] Ahmad Al Badawi et. al. 2022. OpenFHE: Open-Source Fully Homomorphic Encryption Library. Cryptology ePrint Archive, Paper 2022/915. https://eprint.iacr.org/2022/915 https://eprint.iacr.org/2022/915.

[11] Warren Gay. 2018. vcgencmd. *Advanced Raspberry Pi: Raspbian Linux and GPIO Integration* (2018), 337–347.

[12] Robin Geelen and Frederik Vercauteren. 2023. Bootstrapping for BGV and BFV Revisited. *Journal of Cryptology* 36, 2 (2023), 12.

[13] Craig Gentry. 2009. Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*. 169–178.

[14] Craig Gentry. 2021. A Decade (or so) of fully homomorphic encryption. https://www.youtube.com/watch?v=487AjvFW1lk.

[15] Ognjen Glamočanin, Hajira Bazaz, Mathias Payer, and Mirjana Stojilović. 2023. Temperature impact on remote power side-channel attacks on shared fpgas. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.

[16] Charles Gouert, Dimitris Mouris, and Nektarios Georgios Tsoutsos. 2023. SoK: New Insights into Fully Homomorphic Encryption Libraries via Standardized Benchmarks. *Proceedings on Privacy Enhancing Technologies* 2023, 3 (July 2023), 154–172. https://doi.org/10.56553/popets-2023-0075

[17] Inferati Inc. 2021. Introduction to the BFV FHE Scheme. (2021). https://www.inferati.com/blog/fhe-schemes-bfv

[18] Inferati Inc. 2022. Introduction to the BGV FHE Scheme. (2022). https://www.inferati.com/blog/fhe-schemes-bgv

[19] Inferati Inc. 2022. Introduction to the CKKS/HEAAN FHE Scheme. (2022). https://www.inferati.com/blog/fhe-schemes-ckks

[20] Asma Jahangeer, Sibghat Ullah Bazai, Saad Aslam, Shah Marjan, Muhammad Anas, and Sayed Habibullah Hashemi. 2023. A Review on the Security of IoT Networks: From Network Layer's Perspective. *IEEE Access* (2023).

[21] Lei Jiang and Lei Ju. 2022. FHEBench: Benchmarking Fully Homomorphic Encryption Schemes. arXiv:2203.00728 [cs.CR] https://arxiv.org/abs/2203.00728

[22] Andrey Kim, Yuriy Polyakov, and Vincent Zucca. 2021. Revisiting homomorphic encryption schemes for finite fields. In *Advances in Cryptology–ASIACRYPT 2021: 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part III 27*. Springer, 608–639.

[23] Joon-Woo Lee, HyungChul Kang, Yongwoo Lee, Woosuk Choi, Jieun Eom, Maxim Deryabin, Eunsang Lee, Junghyun Lee, Donghoon Yoo, Young-Sik Kim, et al. 2022. Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. *iEEE Access* 10 (2022), 30039–30054.

[24] Yongwoo Lee, Daniele Micciancio, Andrey Kim, Rakyong Choi, Maxim Deryabin, Jieun Eom, and Donghoon Yoo. 2023. Efficient FHEW bootstrapping with small evaluation keys, and applications to threshold homomorphic encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 227–256.

[25] Goran Đorđević, Milan Marković, and Pavle Vuletić. 2022. EVAULATION OF HOMOMORPHIC ENCRYPTION IMPLEMENTATION ON IOT DEVICE. *Journal of Information Technology & Applications* 12, 1 (2022).

[26] H Manohar Reddy, Sajimon P C, and Sriram Sankaran. 2022. On the Feasibility of Homomorphic Encryption for Internet of Things. In *2022 IEEE 8th World Forum on Internet of Things (WF-IoT)*. 1–6. https://doi.org/10.1109/WF-IoT54382.2022.10152214

[27] Iqbal H Sarker, Asif Irshad Khan, Yoosef B Abushark, and Fawaz Alsolami. 2023. Internet of things (iot) security intelligence: a comprehensive overview, machine learning solutions and research directions. *Mobile Networks and Applications* 28, 1 (2023), 296–312.

[28] SEAL 2023. Microsoft SEAL (release 4.1). https://github.com/Microsoft/SEAL. Microsoft Research, Redmond, WA..

[29] Jonathan Takeshita, Nirajan Koirala, Colin McKechney, and Taeho Jung. 2024. HEProfiler: An In-Depth Profiler of Approximate Homomorphic Encryption Libraries. Cryptology ePrint Archive, Paper 2024/1059. https://eprint.iacr.org/2024/1059

[30] Vishal A Thakor, Mohammad Abdur Razzaque, and Muhammad RA Khandaker. 2021. Lightweight cryptography algorithms for resource-constrained IoT devices: A review, comparison and research opportunities. *IEEE Access* 9 (2021), 28177–28193.